

Rescue Video Analyzer

Detección de personas en vídeo sobre una arquitectura **MPI + CUDA**

Trabajo final

Diseño y evaluación de una solución paralela para análisis de vídeo con **MPI + CUDA**.

Idea fuerza: la detección de personas se usa como vehículo para estudiar reparto del trabajo, comunicación mínima y límites reales de escalado.

Arquitecturas
Paralelas

MPI + CUDA
ONNX Runtime CUDA

23 abril 2026

Problema y contexto

- El problema funcional es **detectar personas frame a frame** y producir una salida útil.
- El problema de sistemas es **procesar vídeo con suficiente rendimiento** sin perder orden ni trazabilidad.
- La solución debe aprovechar GPU, repartir trabajo entre procesos y medir qué parte realmente escala.

Lectura del problema

El reto no es solo detectar personas, sino producir una salida ordenada, medible y reproducible sobre vídeo real.

Contexto

Vídeo real

Entrada compartida,
procesamiento distribuido y salidas
auditaables.

Objetivo del trabajo

Objetivo del trabajo

Diseñar y evaluar una arquitectura paralela coherente para análisis de vídeo. El detector es el motor funcional; la aportación principal es cómo se organiza y ejecuta el pipeline.

Entrada real

Vídeos completos

No imágenes sueltas, sino secuencias con cientos de frames y salida final ordenada.

Restricción clave

1 GPU física

El escalado MPI convive con un recurso crítico compartido.

Resultado esperado

Auditable

Cada frame deja tiempos, conteos y trazas exportables.

Resumen ejecutivo

Escalado observado

$\sim 1,30\times$

Speedup relativo frente a 1 worker CUDA en el mejor caso medido sobre una sola GPU física.

Decisión clave

0 frames crudos por MPI

El master reparte rangos de frames; los workers abren el vídeo por su cuenta.

Cuello de botella

70.4 % inferencia

En el caso base, la detección domina claramente el tiempo medio por frame.

Trazabilidad

3 artefactos

Vídeo anotado, CSV por frame y resumen JSON del experimento.

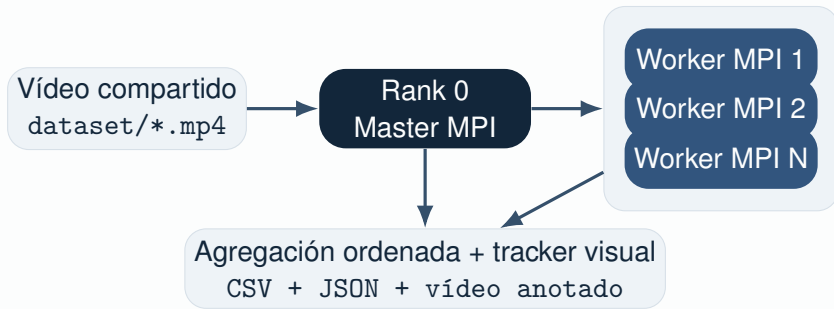
Lectura principal

La arquitectura reduce comunicación, mide bien el trabajo y escala hasta que aparece el límite esperado:

Contexto

la contención sobre la única GPU disponible

Arquitectura global del sistema



Qué reparte el master

No envía imágenes. Solo asigna `start_frame` y `frame_count`, y luego fusiona resultados ordenados.

Qué hace cada worker

Abre el vídeo, lee su lote, preprocesa en CUDA, ejecuta ONNX Runtime CUDA y devuelve un paquete compacto.

Descomposición y comunicación

Descomposición

Paralelismo por datos

El vídeo se divide en lotes de frames; la granularidad viene dada por `batch_size`.

Comunicación

Cabeceras, no payloads

Se evita transferir imágenes BGR por MPI, reduciendo tráfico y presión sobre memoria.

Por qué importa esta elección

Si el master enviara un batch de 8 frames a 1920×1080 en BGR, movería aproximadamente **47.5 MiB** por lote. Aquí solo se envían cabeceras y resultados compactos, así que la comunicación deja de ser el factor dominante.

Afinidad y planificación

Afinidad

GPU por rango local

Cada worker se liga automáticamente a una GPU visible distinta usando su `local worker rank`.

Planificación

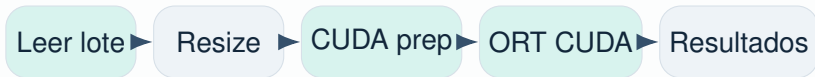
3 schedulers

`dynamic`, `static-contiguous` y `static-round-robin`.

Lectura de diseño

La afinidad por GPU evita colisiones entre workers y permite comparar políticas de reparto sin mezclar el efecto del scheduler con el de la asignación de dispositivos.

Pipeline interno de un worker



Camino de datos eficiente

`preprocess_frame_cuda` genera en GPU el tensor de entrada ya normalizado; esa memoria entra directamente en ONNX Runtime CUDA, evitando copias adicionales innecesarias.

Coste base del pipeline

Caso base np=2, dynamic

Etapa	ms	peso
Lectura	2.91	13.0 %
Preproceso	3.69	16.5 %
Inferencia	15.73	70.4 %

Tiempo total

22.33 ms/frame

Benchmark sobre dataset/videoset.mp4
sin generar vídeo anotado.

Lectura directa

En el caso base, la inferencia domina claramente el tiempo medio por frame: mover datos y preprocesar cuesta bastante menos que ejecutar el detector.

Trazabilidad y salida

- `frame_results.csv`: tiempos por etapa, worker, GPU, conteos y timestamp.
- `summary.json`: métricas agregadas, balance de carga y tiempos globales.
- `annotated_people.mp4`: evidencia visual con cajas y overlay final.

Idea paralela importante

La salida está fuera del tramo distribuido. Por eso el benchmark de escalado desactiva el vídeo anotado: así aísla scheduler y GPU.

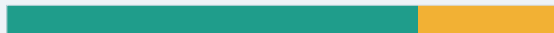
3 artefactos

CSV
JSON
Vídeo anotado

La salida conserva tiempos por etapa, estado del worker y evidencia visual final.

Parte serial del pipeline

Caso funcional con vídeo anotado activado



75.3 % procesamiento distribuido (4986,5 ms)

24.7 % escritura final y anotación (1637,6 ms)

Lectura correcta

Separar compute y salida

Si se mezcla anotación con escalado, el speedup queda sesgado por una parte serial ajena al scheduler.

Diseño experimental

- **Hardware:** Intel Core i7-13700H (20 hilos lógicos) y NVIDIA GeForce RTX 4070 Laptop GPU de 8 GiB.
- **Vídeo de benchmark:**
dataset/videoset.mp4, 341 frames, 640×360 , 25 FPS.
- **Configuración fija:** batch_size=8, models/yolo11s_1088.onnx, sin vídeo anotado.
- **Lanzamientos:** mpirun -np 2, -np 3 y -np 4, equivalentes a 1, 2 y 3 workers CUDA.

Condición del experimento

La comparación solo es útil si se mantiene fijo el detector, el lote de frames y el scheduler. Así el efecto medido es el reparto del trabajo, no la variación del modelo.

Métricas evaluadas

Magnitudes evaluadas

T_p : tiempo total de pared

$S(p) = T_1 / T_p$: speedup relativo

$E(p) = S(p)/p$: eficiencia paralela

Balance

máx / media

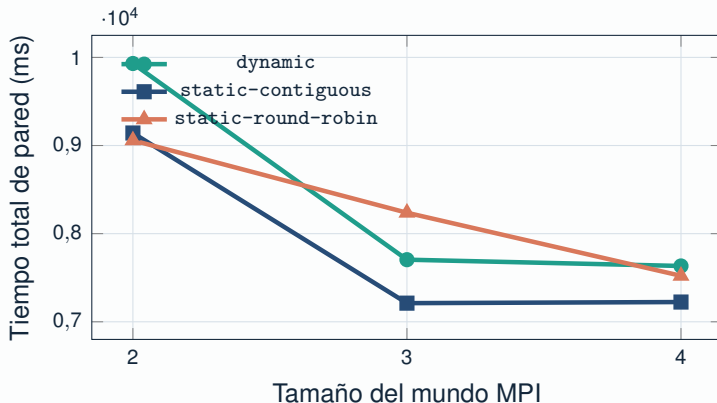
Se usa la razón

`max_to_mean_ratio`
exportada en
`summary.json`.

Lectura correcta

Para comparar speedup, el baseline T_1 se toma manteniendo fija la política de planificación. Si cambia el scheduler, ya no estamos midiendo exactamente el mismo experimento.

Escalado con más workers CUDA



Mejor caso

7211.5 ms

np=3 con
static-contiguous.

Reducción

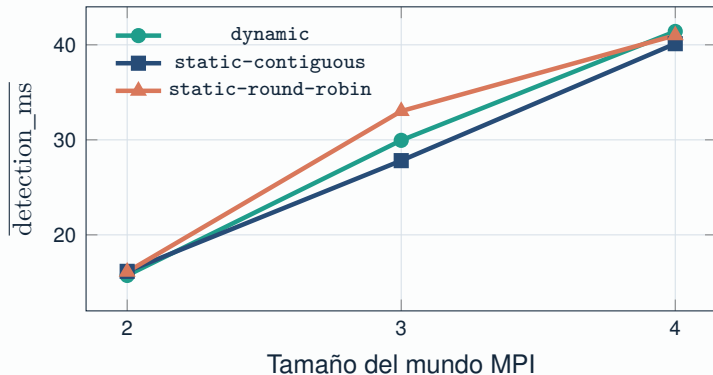
-21.1 %

Respecto a np=2 en el
mismo scheduler.

Lectura del gráfico

Pasar de 1 a 2 workers CUDA mejora el tiempo total; añadir un tercer worker ya casi no aporta beneficio porque la GPU compartida empieza a saturarse.

Contención de GPU y eficiencia paralela



Eficiencia a 2 workers

0.63–0.64

Ya aparece pérdida de eficiencia frente al ideal.

Eficiencia a 3 workers

0.40–0.43

La tercera hebra MPI añade poca mejora real.

Conclusión

El límite no está en MPI, sino en que todos los workers compiten por la misma GPU física durante la Evaluación inferencia.

Scheduler y balance de carga

np	dyn	stat-contig	rr
3	168–173 (1.015)	170–171 (1.003)	168–173 (1.015)
4	112–117 (1.029)	113–114 (1.003)	112–117 (1.029)

Rango de frames por worker; entre paréntesis,
max_to_mean_ratio.

Variabilidad del vídeo

Baja

En `videoset.mp4`, σ de personas/frame
= 2,59.

Correlación

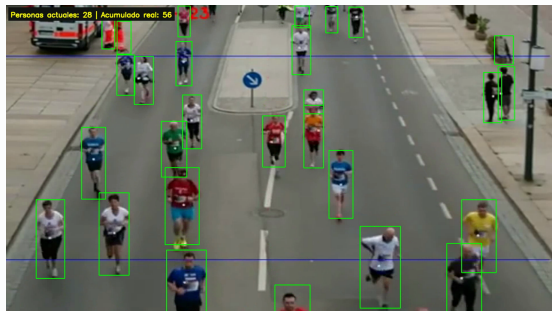
0.087

El número de personas por frame apenas
explica la latencia de inferencia.

Interpretación

La carga por frame es homogénea; por
eso aquí gana `static-contiguous`. El

Resultado funcional del pipeline

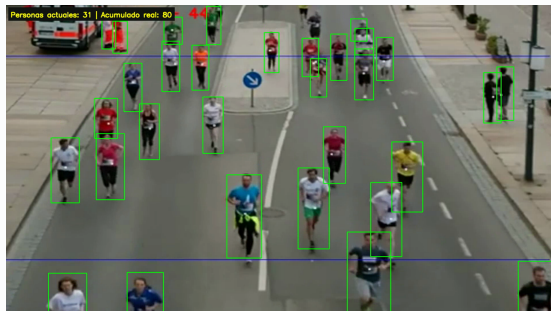


Vídeo anotado

Overlay útil

Muestra personas actuales y acumulado real estimado mediante tracking visual.

Evaluación



Caso mostrado

153 frames

En videoset2.mp4: 29.86 personas/frame de media y 126 personas acumuladas.

17/21

Ejecución en granja de GPUs

Sección pendiente de completar

A fecha **23 de abril de 2026** no se ha tenido acceso experimental a la granja de GPUs. Por tanto, esta parte debe quedar explícitamente marcada como **trabajo pendiente de validación**, no como resultado cerrado.

Lo ya preparado en el código

Afinidad automática por `local worker rank`, rutas compartidas para vídeo/modelo y reparto por workers MPI en nodos multi-GPU.

Qué quedará por medir cuando haya acceso

- Escalado inter-nodo real.
- Coste del filesystem compartido.
- Afinidad GPU por nodo y hostfile MPI.
- Throughput agregado y eficiencia global.
- Sensibilidad a la contención de red y E/S.

Limitaciones

Límite principal

GPU compartida

La inferencia crece en latencia cuando aumenta el número de workers.

Tracker

Visual-only

Sirve para estabilizar la anotación, no para identidad robusta a largo plazo.

Cola serial

Anotación y escritura

La exportación del vídeo final introduce una fracción no paralelizable.

Siguiente paso natural

Validación multi-GPU

La evolución lógica es contrastar el mismo diseño en una granja real.

Lectura de las limitaciones

El siguiente experimento relevante no es cambiar el detector, sino medir scheduler, afinidad y balance de carga cuando la GPU deja de ser un recurso único y pasa a distribuirse entre nodos.

Cierre

Trabajo futuro

Tracker

Visual-only

Sirve para estabilizar la anotación, no para identidad robusta a largo plazo.

Siguiente paso natural

Validación multi-GPU

La evolución lógica es contrastar el mismo diseño en una granja real.

Qué medir después

Cuando haya acceso a la granja, el foco debe pasar a escalado inter-nodo, afinidad real por GPU, impacto del filesystem compartido y sensibilidad a la contención de red.

Conclusiones

- La solución resuelve el problema funcional y, además, deja un pipeline bien medido y reproducible.
- En Arquitecturas Paralelas, lo relevante aquí es **descomposición, comunicación, scheduler MPI y contención sobre GPU**.
- El speedup existe al pasar de 1 a 2 workers CUDA, pero no escala linealmente sobre una sola GPU física.
- En el entorno medido, `static-contiguous` rinde mejor porque la carga por frame es bastante homogénea.
- La validación en granja de GPUs queda abierta como siguiente fase experimental.

Mensaje final

El trabajo demuestra que la calidad de una solución paralela no depende solo de “usar GPU”, sino de **cómo se organiza el sistema, qué se mide y dónde aparecen sus límites reales**.